rqbit 피어 목록 처리 및 관리 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 피어 목록 처리 및 관리 시스템을 상세히 분석합니다. 트래커로부터 받은 피어 정보를 어떻게 파싱하고, 저장하며, 품질을 평가하여 연결을 관리하는지 살펴봅니다.

Compact 피어 목록 형식 분석

1. CompactSerialize 트레이트

rqbit은 효율적인 피어 목록 처리를 위해 compact 형식을 지원합니다:

```
pub trait CompactSerialize: Sized {

type Slice: AsRef<[u8]>;



fn expecting() -> &'static str;

fn as_slice(&self) -> Self::Slice;

fn from_slice_unchecked_len(buf: &[u8]) -> Self;

fn from_slice(buf: &[u8]) -> Option<Self>;

}



pub trait CompactSerializeFixedLen {

fn fixed_len() -> usize;
```

```
}
```

2. IPv4 주소 처리

SocketAddrV4 구현

```
impl CompactSerialize for SocketAddrV4 {
type Slice = [u8; 6];

fn as_slice(&self) -> Self::Slice {
let mut data = [0u8; 6];
data[..4].copy_from_slice(&self.ip().octets()); // IP 주소 4바이트
data[4..6].copy_from_slice(&self.port().to_be_bytes()); // 포트 2바이트
data
}

fn from_slice_unchecked_len(buf: &[u8]) -> Self {
let ip = Ipv4Addr::from_slice_unchecked_len(&buf[..4]);
let port = u16::from_be_bytes([buf[4], buf[5]]);
SocketAddrV4::new(ip, port)
}

fn expecting() -> &'static str {
"6 bytes for SocketAddrV4"
```

```
}
}

impl CompactSerializeFixedLen for SocketAddrV4 {
fn fixed_len() -> usize {
6 // 4바이트 IP + 2바이트 포트
}
}
```

3. IPv6 주소 처리

SocketAddrV6 구현

```
impl CompactSerialize for SocketAddrV6 {
type Slice = [u8; 18];

fn as_slice(&self) -> Self::Slice {
let mut data = [0u8; 18];
data[..16].copy_from_slice(&self.ip().octets()); // IP 주소 16바이트
data[16..18].copy_from_slice(&self.port().to_be_bytes()); // 포트 2바이트
data
}

fn from_slice_unchecked_len(buf: &[u8]) -> Self {
```

```
let ip = Ipv6Addr::from_slice_unchecked_len(&buf[..16]);
let port = u16::from_be_bytes([buf[16], buf[17]]);
SocketAddrV6::new(ip, port, 0, 0) // flow_info와 scope_id는 0으로 설정
}

fn expecting() -> &'static str {
"18 bytes for SocketAddrV6"
}
}

impl CompactSerializeFixedLen for SocketAddrV6 {
fn fixed_len() -> usize {
18 // 16바이트 IP + 2바이트 포트
}
}
```

4. CompactListInBuffer 구조체

피어 목록을 효율적으로 처리하기 위한 컨테이너:

```
pub struct CompactListInBuffer<Buf, T> {
buf: Buf, // 원시 바이트 데이터
_phantom: PhantomData<T>, // 타입 정보
}
```

```rust
impl<Buf, T> CompactListInBuffer<Buf, T>
where
Buf: AsRef<[u8]>,
T: CompactSerialize + CompactSerializeFixedLen,
{
pub fn new_from_buf(buf: Buf) -> Self {
Self {
buf,
_phantom: Default::default(),
}
}

pub fn iter(&self) -> impl Iterator<Item = T> + Clone {
self.buf
.as_ref()
.chunks_exact(T::fixed_len()) // 고정 길이로 청크 분할
.map(|chunk| T::from_slice_unchecked_len(chunk))
}

pub fn get(&self, idx: usize) -> Option<T> {
let offset = idx * T::fixed_len();
let end = offset + T::fixed_len();
```

```
let b = self.buf.as_ref().get(offset..end)?;

Some(T::from_slice_unchecked_len(b))

}

}
```

주요 특징:

- 제로카피 파싱: 원본 버퍼를 복사하지 않고 직접 참조
- 고정 길이 최적화: 각 항목의 크기가 고정되어 있어 빠른 인덱싱 가능
- 타입 안전성: 컴파일 타임에 타입 검증

피어 정보 저장 및 관리

1. PeerStates 구조체

피어 상태를 중앙에서 관리하는 구조체:

```
pub(crate) struct PeerStates {

pub session_stats: Arc<AggregatePeerStatsAtomic>,

pub live_outgoing_peers: RwLock<HashSet<PeerHandle>>, // PEX용 활성 피어

pub stats: AggregatePeerStatsAtomic, // 통계 정보

pub states: DashMap<PeerHandle, Peer>, // 피어 상태 맵

}
```

2. 피어 추가 및 중복 제거

add_if_not_seen 메서드

```rust
pub fn add_if_not_seen(&self, addr: SocketAddr) -> Option<PeerHandle> {
use dashmap::mapref::entry::Entry;
match self.states.entry(addr) {
Entry::Occupied(_) => None, // 이미 존재하는 피어
Entry::Vacant(vac) => {
vac.insert(Peer::new_with_outgoing_address(addr));
atomic_inc(&self.stats.queued);
atomic_inc(&self.session_stats.queued);
atomic_inc(&self.stats.seen);
atomic_inc(&self.session_stats.seen);
Some(addr) // 새로운 피어 추가됨
}
}
}
```

중복 제거 메커니즘:

- `DashMap` 을 사용한 동시성 안전한 피어 관리
- 주소 기반 중복 검사
- 원자적 통계 업데이트

3. 피어 상태 관리

Peer 구조체와 상태 전이

```rust
pub enum PeerState {
Queued, // 대기 중
Connecting, // 연결 중
Live(LivePeerState), // 활성 상태
Dead, // 연결 실패/종료
NotNeeded, // 더 이상 필요하지 않음
}

pub struct LivePeerState {
pub peer_interested: bool, // 피어가 우리에게 관심 있는지
pub bitfield: BF, // 피어가 가진 조각들
pub tx: PeerTx, // 메시지 전송 채널
pub inflight_requests: Vec<ChunkRequest>, // 진행 중인 요청들
pub connection_kind: ConnectionKind, // 연결 타입
pub peer_id: Id20, // 피어 ID
}
```

4. 피어 품질 평가 시스템

피어 통계 수집

```rust
pub struct PeerCountersAtomic {
pub incoming_connections: AtomicU64, // 인바운드 연결 수
pub outgoing_connection_attempts: AtomicU64, // 아웃바운드 연결 시도
```

```
pub times_i_stole: AtomicU64, // 다른 피어로부터 훔친 횟수
pub times_stolen_from_me: AtomicU64, // 나에게서 훔겨간 횟수
pub downloaded_bytes: AtomicU64, // 다운로드한 바이트
pub uploaded_bytes: AtomicU64, // 업로드한 바이트
pub downloaded_pieces: AtomicU64, // 다운로드한 조각 수
}
```

백오프 전략

```
pub struct PeerStats {
pub counters: Arc<PeerCountersAtomic>,
backoff: RwLock<ExponentialBackoff>,
}

impl PeerStats {
pub fn reset_backoff(&self) {
*self.backoff.write() = ExponentialBackoff::default();
}

pub fn next_backoff(&self) -> Duration {
self.backoff.write().next_backoff()
}
```

```
}
```

5. 피어 선택 알고리즘

연결 제한 관리

```
// TorrentStateLive에서
peer_semaphore: Arc<Semaphore>, // 동시 연결 수 제한 (기본 128개)


// 피어 추가 시
let permit = state.peer_semaphore.clone().acquire_owned().await?;
```

피어 우선순위 결정 요소

1. **연결 가능성**

- 백오프 시간이 지났는지 확인
- 이전 연결 실패 횟수 고려

2. **유용성 평가**

```
pub fn is_peer_not_interested_and_has_full_torrent(
&self,
handle: PeerHandle,
total_pieces: usize,
) -> bool {
self.with_live(handle, |live| {
!live.peer_interested && live.has_full_torrent(total_pieces)
```

```
})
.unwrap_or(false)
}
```

3. **블록리스트 확인**

```
let is_in_blocklist = session.blocklist.is_blocked(addr.ip());
if is_in_blocklist {
session.stats.counters.blocked_outgoing.fetch_add(1, Ordering::Relaxed);
debug!(?addr, "blocked outgoing connection");
continue;
}
```

6. 피어 연결 관리

피어 큐 시스템

```
// 피어 추가 큐
peer_queue_tx: UnboundedSender<SocketAddr>,
```

```
pub(crate) fn add_peer_if_not_seen(&self, addr: SocketAddr) -> crate::Result<boo
match self.peers.add_if_not_seen(addr) {
Some(handle) => handle,
None => return Ok(false), // 이미 알려진 피어
```

```
};

// 큐에 추가하여 연결 시도
self.peer_queue_tx
.send(addr)
.ok()
.ok_or(Error::TorrentIsNotLive)?;
Ok(true)
}
```

피어 연결 태스크

```
async fn task_peer_adder(
self: Arc<Self>,
mut peer_queue_rx: UnboundedReceiver<SocketAddr>,
) -> crate::Result<()> {
loop {
let addr = peer_queue_rx.recv().await.ok_or(Error::TorrentIsNotLive)?;
// 완료된 토렌트에서 업로드 비활성화 시 피어 무시
if state.shared.options.disable_upload() && state.is_finished_and_no_active_stre
debug!(?addr, "ignoring peer as we are finished");
state.peers.mark_peer_not_needed(addr);
continue;
}
```

```
// 블록리스트 확인
let session = state.shared.session.upgrade().ok_or(Error::SessionDestroyed)?;
let is_in_blocklist = session.blocklist.is_blocked(addr.ip());
if is_in_blocklist {
session.stats.counters.blocked_outgoing.fetch_add(1, Ordering::Relaxed);
debug!(?addr, "blocked outgoing connection");
continue;
}

// 세마포어로 동시 연결 수 제한
let permit = state.peer_semaphore.clone().acquire_owned().await?;
// 피어 관리 태스크 생성
state.spawn(
debug_span!(parent: state.shared.span.clone(), "manage_peer", peer = ?addr),
format!("[{}][addr={addr}]manage_peer", state.shared.id),
state.clone().task_manage_outgoing_peer(addr, permit),
);
}
}
```

7. 피어 성능 모니터링

실시간 통계 수집

```
pub struct AggregatePeerStats {
pub seen: u64, // 발견된 총 피어 수
pub connecting: u64, // 연결 중인 피어 수
pub live: u64, // 활성 피어 수
pub dead: u64, // 죽은 피어 수
pub not_needed: u64, // 불필요한 피어 수
pub queued: u64, // 대기 중인 피어 수
pub steals: u64, // 조각 훔치기 횟수
}
```

피어별 성능 스냅샷

```
pub fn per_peer_stats_snapshot(&self, filter: PeerStatsFilter) -> PeerStatsSnaps
PeerStatsSnapshot {
peers: self
.peers
.states
.iter()
.filter(|e| filter.state.matches(e.value().get_state()))
.map(|e| (e.key().to_string(), e.value().into()))
.collect(),
}
```

```
}
```

8. 조각 훔치기 메커니즘

피어 간 조각 재할당

```
pub(crate) fn on_steal(
&self,
from_peer: SocketAddr,
to_peer: SocketAddr,
stolen_idx: ValidPieceIndex,
) {
// 통계 업데이트
self.with_peer(to_peer, |p| {
atomic_inc(&p.stats.counters.times_i_stole);
});
self.with_peer(from_peer, |p| {
atomic_inc(&p.stats.counters.times_stolen_from_me);
});
self.stats.inc_steals();
self.session_stats.inc_steals();

// 기존 요청 취소
self.with_live_mut(from_peer, "send_cancellations", |live| {
```

```
let to_remove = live
.inflight_requests
.iter()
.filter(|r| r.piece_index == stolen_idx)
.copied()
.collect::<Vec<_>>();
for req in to_remove {
let _ = live
.tx
.send(WriterRequest::Message(Message::Cancel(Request {
index: stolen_idx.get(),
begin: req.offset,
length: req.size,
})));
}
});
}
```

9. 메모리 효율성 최적화

SmallSlice 구조체

```
pub struct SmallSlice<const N: usize> {
data: [u8; N],
```

```
len: usize,
}

impl<const N: usize> SmallSlice<N> {
pub fn new_from_buf(buf: &[u8]) -> Self {
let mut s = Self::new();
s.extend(buf);
s
}

pub fn extend(&mut self, buf: &[u8]) {
self.data[self.len..self.len + buf.len()].copy_from_slice(buf);
self.len += buf.len();
}
}
```

최적화 특징:

- 스택 할당을 통한 힙 할당 최소화
- 가변 길이 데이터의 효율적 저장
- 제로카피 연산 지원

10. 동시성 및 스레드 안전성

DashMap 활용

```
pub states: DashMap<PeerHandle, Peer>, // 동시성 안전한 해시맵
```

원자적 통계 업데이트

```
fn atomic_inc(counter: &AtomicU64) {
counter.fetch_add(1, Ordering::Relaxed);
}
```

읽기-쓰기 락 최적화

```
pub fn with_peer_mut<R>(
&self,
addr: PeerHandle,
reason: &'static str,
f: impl FnOnce(&mut Peer) -> R,
) -> Option<R> {
use crate::torrent_state::utils::timeit;
timeit(reason, || self.states.get_mut(&addr))
.map(|e| f(TimedExistence::new(e, reason).value_mut()))
}
```

결론

rqbit의 피어 목록 처리 및 관리 시스템은 다음과 같은 고급 기능들을 제공합니다:

1. **효율적인 Compact 형식 지원**: 제로카피 파싱과 고정 길이 최적화

2. **지능적인 중복 제거**: 주소 기반 중복 검사와 상태 추적

3. **동적 피어 품질 평가**: 성능 통계와 백오프 전략

4. **확장 가능한 연결 관리**: 세마포어 기반 동시 연결 제한

5. **실시간 성능 모니터링**: 원자적 통계 수집과 스냅샷

6. **조각 훔치기 최적화**: 피어 간 효율적인 작업 재분배

7. **메모리 효율성**: 스택 할당과 제로카피 최적화

8. **동시성 안전성**: DashMap과 원자적 연산 활용

이러한 구현을 통해 대규모 피어 네트워크에서도 효율적이고 안정적인 피어 관리를 보장합니다.